

M12 / LEARNER BOOK

Distributed processing with Spark

Standalone Zero-to-Job-Ready Data Engineer 2026

18 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE How to use this module	3
DE12-01 Why Spark exists: driver, executors, partitions and tasks	4
DE12-02 PySpark setup, SparkSession and laptop-safe local mode	7
DE12-03 DataFrames and explicit schemas	10
DE12-04 Spark SQL and expression equivalence	13
DE12-05 Lazy transformations and actions	16
DE12-06 Jobs, stages, tasks and physical plans	19
DE12-07 repartition vs coalesce	22
DE12-08 Aggregations and shuffle cost	25
DE12-09 Joins and broadcast strategy	28
DE12-10 Join correctness, fan-out and reconciliation	31
DE12-11 Data skew	34
DE12-12 Adaptive Query Execution (AQE)	37
DE12-13 Caching and persistence	40
DE12-14 Parquet I/O and partition pruning	43
DE12-15 Built-in functions vs Python UDFs	46
DE12-16 Driver safety: collect, toPandas and bounded inspection	49
DE12-17 Testing and debugging Spark transformations	52
DE12-18 Integrated PySpark pipeline and distributed-processing checkpoint	55

START HERE

How to use this module

Open the matching practice folder from the course root: 11_M12/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

This book is the teacher. M12 uses a tiny inspectable fixture so you can prove business correctness while learning real Spark 4.2.0 APIs and distributed execution reasoning. Small data here is intentional; it is not a claim that Spark is needed for 18 production rows.

Runtime honesty: Python reference/simulator controls and static Spark-code checks pass, but live Spark runtime is a separate proof. Collect

SparkSession/UI/physical-plan/Parquet/cache/broadcast/AQE evidence on the learner PC.

Practice: 11_M12/practice/ Quick revision:

12_REVISION_AND_TESTS/M12/M12_Quick_Revision_Book.pdf Protected answers (open only after saved attempt): 12_REVISION_AND_TESTS/M12/PROTECTED_OPEN_AFTER_ATTEMPT/M12_Module_Revision_Test_Answers.pdf

DE12-01

Why Spark exists: driver, executors, partitions and tasks

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Driver, executors, partitions, distributed work mental model

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

M11 taught how analytical files and table metadata are arranged. M12 answers the next question: what happens when one process is no longer enough to scan, join and aggregate the data comfortably? Spark separates coordination from parallel task execution. The point is not 'big data magic'; it is a concrete execution model you can reason about when work is divided across partitions.

Picture it in your head

Picture a restaurant pass. The driver is the head coordinator reading the order tickets and deciding the plan. Executors are kitchens that do assigned work. A partition is a tray of records given to one task at a time. A task is one unit of work over one partition for one stage. In local[2] the whole restaurant is still your laptop: two local task threads can work concurrently, but you have not created two production worker machines.

Words before we use them

driver = application coordinator; executor = process that runs tasks and stores data for an application; partition = chunk of distributed data; task = unit of work for one partition in one stage; cluster manager = service that allocates resources in a real cluster; local[2] = local master with two worker threads.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Driver, executors, partitions, distributed work mental model [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark 4.2.0 DataFrame quickstart Structured distributed API orientation [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/quickstart_df.html

Follow with me once

Run the pure-Python architecture simulator first so the terms are stable before Spark is installed. Then, on the learner PC, run `01_architecture_partitions.py`. Record `defaultParallelism`, the input partition count and row count. The 18-row dataset is intentionally tiny: we are learning the execution model, not proving Spark is faster than Python on 18 rows.

```
python practice/simulators/distributed_reasoning_simulator.py --section architecture
# learner PC with PySpark:
cd practice/spark
python 01_architecture_partitions.py
```

What you should see

The simulator says the driver plans/co-ordinates, executors run tasks over partitions, and `local[2]` is two local task threads. The Spark lab should report 18 source rows. Partition counts can differ by reader and environment; the lesson is to inspect them rather than memorise a number.

Why it worked

Spark can parallelize because the logical dataset is split into partitions and tasks can operate on independent partitions. The driver does not personally process every row just because Python started the application. Keeping this distinction prevents later errors such as bringing all data back to the driver with `collect()`.

Break it / common failure

Do not say 'a partition is a server' or 'one row equals one task.' One executor can process many partitions over time, and one partition can contain many rows. Also do not infer a production cluster from `local[2]`. Local mode is a useful execution engine on one machine, not distributed hardware.

Reason through a realistic incident

A learner sees four tasks in the Spark UI and says there must be four executors. Diagnose the claim. Ask which stage ran, how many partitions it had, what master was used, and where executor processes actually live. Task count follows partition work; it is not an executor count.

Deepen the engineering reasoning

The useful abstraction is layered: your `DataFrame` code describes a computation, the driver builds/optimizes a plan, stage boundaries appear where dependencies require data exchange, and tasks execute stage work per partition. Resource allocation and task scheduling are related but not the same thing. This layered model lets you debug from the outside in: first business result, then `DataFrame` plan, then stage/task evidence, then resource symptoms.

Try it yourself

Draw driver -> stage -> tasks -> partitions from memory. Explain what changes and what does not change when the same 18-row job moves from `local[2]` to a real cluster. Name two things that can scale out and two business-correctness rules that must remain identical.

Fresh changed retry

A stage has 80 partitions and the cluster has 8 executor cores available. Explain why 80 tasks do not run simultaneously and why reducing to 8 partitions automatically is not always the right repair.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Use the execution model to explain evidence, not as vocabulary theatre. For every Spark incident be able to state where the code runs, which operation created the partition boundary, what one output row means and which row/key/total check proves business correctness.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-02

PySpark setup, SparkSession and laptop-safe local mode

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Local PySpark setup and first SparkSession context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

A Spark lesson is useless if setup advice encourages an 8 GB laptop to run like a production cluster. The goal is a small, reproducible local lane that exercises the same DataFrame APIs while keeping resource use bounded. Current PySpark 4.2.0 supports Python 3.10+ and requires Java 17+; the course profile uses local[2], 1g driver memory and four shuffle partitions so plans stay visible on a modest machine.

Picture it in your head

Think of a flight simulator rather than an airport. The controls and rules are real, but the scale is intentionally small. SparkSession is your cockpit entry point. local[2] tells Spark to use two local worker threads. The four-shuffle-partition setting makes tiny examples readable; it is not a recommendation that production jobs should always have four partitions.

Words before we use them

SparkSession = structured API entry point; master = execution location such as local[2] or a cluster manager; driver memory = heap available to the driver JVM; shuffle partitions = default number of partitions for many SQL shuffle outputs; AQE = runtime adaptive optimization; JAVA_HOME = environment pointer to Java installation.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Local PySpark setup and first SparkSession context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: PySpark 4.2.0 installation Python 3.10+; Java 17+; pip/venv options [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/install.html

Follow with me once

On the learner PC, create a clean virtual environment and pin pyspark==4.2.0. Run `java -version`, `python --version` and `00_env_probe.py`. Then create the session using `common.py`, print the four settings and stop the session. This review release does not certify live PySpark execution. The learner PC must prove the SparkSession starts and the commands run with the stated settings.

```
python -m venv .venv
# activate .venv, then:
pip install -r practice/spark/requirements.txt
java -version
python practice/spark/00_env_probe.py
```

What you should see

Your learner environment should report PySpark 4.2.0, a supported Python version and Java 17 or later. A SparkSession created through `common.spark_session()` should use `local[2]`, 1g driver memory, `spark.sql.shuffle.partitions=4` and `AQE true`. Record the actual values rather than assuming configuration took effect.

Why it worked

A pinned small environment separates course logic from machine noise. When every lab uses the same session builder, a plan difference is less likely to be caused by one forgotten setting. Explicitly stopping sessions also reduces the chance that multiple JVMs compete for your limited RAM.

Break it / common failure

Common setup mistakes are mixing system Python and venv Python, installing multiple Spark versions, assuming Java 8/11 guidance from an old video is current, or opening Airflow/Docker/PostgreSQL/Spark together on an 8 GB laptop. Another mistake is treating `pip install pyspark` success as proof that a Spark

job actually starts.

Reason through a realistic incident

`import pyspark` works, but `SparkSession` fails with a Java gateway error. Do not rewrite the pipeline. Check `java -version`, `JAVA_HOME`, the active Python environment and PySpark version first. A runtime bootstrap failure happens before your transformation logic is evaluated.

Deepen the engineering reasoning

Version pinning is part of reproducibility, not administrative overhead. Spark spans Python, Py4J and a JVM runtime; failures can originate at multiple layers. Record versions with the same evidence as code. Later modules can choose a different pinned environment when another library has stricter compatibility; do not force one global environment merely for convenience.

Try it yourself

Write a one-page learner-PC Spark setup record: Python version, Java version, PySpark version, master, driver-memory setting, shuffle partitions, AQE value and the exact command that successfully created/stopped a session. Mark anything not executed as pending.

Fresh changed retry

Suppose a tutorial uses local[*] and an 8g driver. Explain why blindly copying it onto your laptop could make the lesson less reliable, and how you would choose a safer teaching configuration without pretending it is production tuning.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

A trustworthy runtime record says what was really executed on which environment. Never turn static code inspection into a live-Spark PASS. Stable setup and honest runtime evidence make later performance observations comparable.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-03

DataFrames and explicit schemas

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

DataFrame reading and schema-oriented workflow

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Spark DataFrames are not giant Python lists. They are distributed, schema-bearing logical datasets whose operations can be inspected and optimized. Explicit schemas matter because engineering pipelines should not repeatedly guess whether qty is text or integer, whether a field is nullable, or whether a malformed value should silently change the inferred type.

Picture it in your head

Picture a warehouse receiving form. Before boxes arrive, the form says order_id is text, order_date is a date, qty is integer and unit_price is decimal-like numeric. A DataFrame carries that contract with the rows. Spark can reason about named typed columns; a Python loop over arbitrary dictionaries hides much of that structure from the optimizer.

Words before we use them

DataFrame = distributed table-like dataset with named columns; schema = column names/types/nullability; StructType/StructField = explicit schema objects; Column expression = symbolic computation evaluated by Spark; inference = reader guesses types from data; Catalyst = optimizer framework behind Spark SQL/DataFrames.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) DataFrame reading and schema-oriented workflow [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: PySpark DataFrame API Structured DataFrame operations and schema surface [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/dataframe.html>

Follow with me once

Open `common.py` and inspect `ORDER_SCHEMA`. Then run `02_dataframe_schema.py`. It reads `orders.csv` with the explicit schema, derives `gross_sales = qty * unit_price`, selects four columns and shows them in deterministic order for inspection. Compare this with reading the same CSV without a schema and ask what assumptions Spark would have to make.

```
cd practice/spark
```

```
python 02_dataframe_schema.py
# inspect common.py before the run
```

What you should see

The schema should show typed date/integer/double fields instead of all strings. The resulting DataFrame keeps one row per source order and adds `gross_sales` without collecting the whole dataset into Python. The source fixture remains 18 rows.

Why it worked

Built-in column expressions remain part of Spark's structured plan. Explicit types let Spark validate operations and choose encodings without first treating everything as Python objects. More importantly, the data contract becomes reviewable: another engineer can see what the pipeline expects before a bad batch reaches a downstream join.

Break it / common failure

Do not assume explicit schema means bad values are automatically quarantined with useful reasons; input error behavior still needs a contract. Do not mark every field nullable just to avoid errors. And do not confuse schema with grain: a perfect schema cannot tell you whether one row is one order, item or event unless you state it.

Reason through a realistic incident

A new file contains `qty='five'`. With loose inference, behavior may vary depending on the reader/data sample. With an explicit integer contract, the mismatch becomes visible at the ingestion boundary. Decide whether the source row should fail, become null under an explicit policy, or be quarantined upstream; do not silently reinterpret the business field.

Deepen the engineering reasoning

Schema is one of three correctness layers in this module: schema says what columns mean and how values are typed; grain says what one row represents; keys/cardinality say how rows may combine. Many Spark bugs happen because engineers validate only one layer. Keep all three in evidence notes, especially before joins and aggregations.

Try it yourself

Create an explicit schema for a fresh shipment file with `shipment_id`, `shipped_at`, `customer_id`, `pieces` and `weight_kg`. State nullability decisions and one bad-value policy. Then write the DataFrame read skeleton without using `inferSchema`.

Fresh changed retry

The partner adds a nullable `carrier_code` column. Explain how you would update the schema deliberately and how you would distinguish compatible additive change from an unexpected rename of `customer_id`.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Prefer explicit schemas at stable ingestion boundaries and built-in expressions for transformations. Save `printSchema()` evidence, row grain and a small bounded sample; the schema alone is not proof that all business values are valid.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-04

Spark SQL and expression equivalence

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

DataFrame/Spark SQL transformation examples

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Spark SQL and the DataFrame API are two ways to describe many of the same structured computations. You should be able to translate between them because teams mix notebooks, SQL models and Python jobs. The important skill is semantic equivalence: same input population, same output grain, same grouping/filter rules and independently matching totals.

Picture it in your head

Picture two languages giving instructions to the same query planner. One says `df.filter(...).groupBy(...)`; the other says `SELECT ... WHERE ... GROUP BY ...`. If both describe the same logic, the engine can optimize them through the same structured subsystem. Syntax familiarity is secondary to proving that the two requests really mean the same thing.

Words before we use them

temporary view = session-scoped name exposing a DataFrame to SQL; expression equivalence = different syntax produces the same intended result; semantic equivalence = same business meaning, not merely similar-looking code; SQL plan = optimizer representation of SQL/DataFrame operations.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)
DataFrame/Spark SQL transformation examples [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Spark DataFrame quickstart Temporary views and SQL/DataFrame examples [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/quickstart_df.html

Follow with me once

Run `03_sql_equivalence.py`. The DataFrame route filters Completed orders, groups by region and sums gross sales. The SQL route queries a temporary view with the same rule. Sort both by region before comparison. Expected totals are East 5950, North 6380, South 6000 and West 6900.

```
cd practice/spark
python 03_sql_equivalence.py
```

What you should see

Both routes produce four regional rows with identical totals; their sum is 25,230. The result grain is one row per region. If one route includes Pending/Cancelled rows or uses a different calculation, matching syntax style is irrelevant because business semantics differ.

Why it worked

DataFrame and SQL expressions enter Spark's structured optimizer. Translating between them makes assumptions visible: WHERE/filter defines the population, groupBy/GROUP BY changes the grain, and aggregate expressions define measures. Sorting is added only to make evidence comparisons deterministic, not because DataFrames are inherently ordered.

Break it / common failure

A common false comparison is `df.show()` versus a SQL query with ORDER BY and assuming row-order difference means value mismatch. DataFrames are not guaranteed ordered unless you request ordering. Another mistake is registering a view before adding `gross_sales`, then writing SQL that assumes the column already exists.

Reason through a realistic incident

The Python job reports West 6900 while an analyst's SQL reports West 7800. Before blaming Spark, compare filters, gross-sales formula, null handling and source version. Use a shared temporary view or small fixed fixture to isolate semantic differences.

Deepen the engineering reasoning

Equivalence is useful for debugging because SQL can make group/filter logic obvious while DataFrame code makes programmatic composition obvious. But duplicated logic can drift. In production, choose a clear ownership boundary rather than maintaining two hand-copied implementations forever; use equivalence tests during migration or review.

Try it yourself

Write both DataFrame and Spark SQL versions of Completed gross sales by category. Expected Accessories 16830 and Office 8400. State the result grain and one method to compare results independent of display order.

Fresh changed retry

Rewrite a DataFrame filter for orders on/after 2026-09-03 into SQL. Then explain which part is business logic and which part only controls output ordering.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Be bilingual enough to review both APIs, but validate business equivalence with row counts, keys and totals. Matching screenshots are weak evidence; deterministic comparison of values and schema is stronger.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-05

Lazy transformations and actions

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Transformation chain and action context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Spark DataFrame transformations are lazy: constructing a filter/select/groupBy chain usually builds a plan rather than immediately scanning the full source. An action such as count, show, collect, write or first asks Spark to execute enough of that plan to produce a result. This matters because repeated actions can repeat work unless persistence or prior output changes the execution path.

Picture it in your head

Imagine writing a recipe on a whiteboard. Adding 'filter Completed' and 'group by region' changes the recipe, but the kitchen does not cook yet. `show()` is the order bell. Spark now turns the recipe into an executable plan and schedules jobs. A second bell can cause another execution if results were not persisted or materialized elsewhere.

Words before we use them

transformation = returns a new logical dataset; action = requests a result or side effect that triggers execution; lazy evaluation = defer execution until needed; lineage = dependency chain used to derive data; action boundary = point where computation must produce something observable.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Transformation chain and action context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark DataFrame quickstart Lazy DataFrame transformations and actions context [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/quickstart_df.html

Follow with me once

Run `04_lazy_plans.py`. The script creates a filter -> withColumn -> groupBy chain, prints a message before any action, calls `explain('formatted')`, then uses `show()`. Notice that `explain` inspects the plan while `show` requests output. Later, use the Spark UI on your learner PC to associate actions with jobs rather than memorising a list of method names.

```
cd practice/spark
python 04_lazy_plans.py
```

What you should see

Before `show()`, you have a `DataFrame plan` object, not four already-calculated regional totals sitting in Python memory. The action produces the four totals. The exact number of jobs/stages can depend on Spark/version/operation; inspect runtime evidence rather than promising one fixed count from the source code alone.

Why it worked

Laziness gives Spark room to optimize a whole structured plan: it can push filters, prune columns or change join strategy rather than executing each line as an isolated Python instruction. The cost is conceptual: source code order is not a timeline of physical execution.

Break it / common failure

Do not say 'nothing happens' before an action in an absolute sense: analysis/planning and metadata work can occur, and some operations may interact with catalog/source metadata. The important boundary is that full data computation is deferred. Also do not call `count()` casually after every debugging line; each action can trigger real work.

Reason through a realistic incident

A notebook has five `display/count/show` calls between transformations and runs much slower than a single final write. Explain that the notebook may be triggering multiple executions.

Remove unnecessary

actions, use plan inspection/bounded checks, or cache only if repeated reuse justifies it.

Deepen the engineering reasoning

Action discipline is a reliability skill as much as a performance skill. An action can write output, move data to the driver or materialize cache. Before running it, know whether it is read-only, driver-bound, or side-effecting. Later orchestration should treat output-writing actions as explicit boundaries with idempotency rules.

Try it yourself

Classify these as transformations, actions or context-dependent APIs: `select`, `filter`, `withColumn`, `groupBy` result construction, `count`, `first`, `write.parquet`, `explain`. For each action, state what evidence or side effect it requests.

Fresh changed retry

A teammate adds `df.count()` only to print 'loaded successfully' before every stage. Explain when that check could be useful and when it is redundant expensive work. Propose a more targeted invariant.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Plan first, trigger work deliberately, and record which actions exist. Do not equate Python line count with Spark work. Use explain, Spark UI/runtime metrics and independent result controls together.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-06

Jobs, stages, tasks and physical plans

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Execution-plan and distributed-stage context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

When an action triggers Spark, the driver turns the optimized plan into executable work. Jobs are created by actions; jobs are split into stages around shuffle/exchange boundaries; stages contain tasks, commonly one per partition. `DataFrame.explain()` is your first window into the plan, while the Spark UI on a real runtime shows what actually executed.

Picture it in your head

Picture a road trip with toll bridges. Long stretches of local road can be completed without everyone regrouping. A shuffle boundary is like a bridge where records with the same destination key must be redistributed before the next stretch can start. Each stretch becomes a stage; each lane/partition gets a task.

Words before we use them

job = computation triggered by an action; stage = group of tasks that can run without a shuffle dependency between them; Exchange = physical-plan node often indicating redistribution; physical plan = chosen execution operators; Spark UI = runtime evidence for jobs/stages/tasks/storage/SQL; explain = textual plan inspection.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Execution-plan and distributed-stage context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: `DataFrame.explain` Inspect logical/physical plans instead of guessing [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.explain.html>

Follow with me once

Use `04_lazy_plans.py` and `06_aggregation_shuffle.py`. Run `explain('formatted')` before the action and look for an Exchange around the grouped aggregation. On the learner PC, open the Spark UI while the app is alive and connect the source plan to actual job/stage/task evidence. Save both plan text and a short explanation; a screenshot alone is not reasoning.

```
cd practice/spark
python 06_aggregation_shuffle.py
```

What you should see

The plan should show a distributed aggregation path with an Exchange/shuffle boundary for grouping by region. Runtime stage/task counts depend on partitions and adaptive execution; do not hard-code a universal number into your explanation.

Why it worked

Rows with the same group key can begin in different input partitions. To compute a final per-region sum, Spark must bring compatible key groups together. That cross-partition dependency creates a stage boundary. The physical plan gives a hypothesis; runtime statistics tell you how the actual execution behaved.

Break it / common failure

Do not label every operation a shuffle. Simple select/filter/withColumn can often remain narrow. Conversely, do not assume a groupBy is cheap because the result has only four rows; many source rows may have moved before those four results appeared. Small output is not small work.

Reason through a realistic incident

A job outputs four rows but takes minutes at scale. The analyst says 'only four rows, Spark is slow.' Explain how a four-row aggregation can require scanning billions of rows and shuffling keys. Inspect scan,

Exchange and task metrics instead of reasoning from output size.

Deepen the engineering reasoning

Plans have logical, optimized and physical layers. You do not need to memorise every node. Learn to locate scans, filters, projections, exchanges, joins and aggregates; then ask whether each is expected from the business request. Performance tuning begins only after correctness and expected plan shape are understood.

Try it yourself

Given `read -> filter Completed -> select region, amount -> groupBy region -> sum -> show`, mark the likely narrow work and shuffle boundary. State what you would inspect in explain and what only the runtime UI can confirm.

Fresh changed retry

A query adds `orderBy(gross_sales desc)` after groupBy. Explain why ordering can add another redistribution/sort requirement and why the exact stage count should be observed rather than

guessed.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Use plan evidence as a map, not fortune-telling. Pair `explain('formatted')` with actual runtime metrics and result reconciliation. A faster wrong plan is still wrong.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-07

repartition vs coalesce

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Partition-count changes in practical Spark work

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Partition count controls how work and data are divided, but changing it is not a universal speed knob. `repartition()` can increase or decrease partitions and typically redistributes data; `coalesce()` is commonly used to reduce partitions with a narrower dependency when possible. Both can be useful, and both can be harmful when used without file-size/workload evidence.

Picture it in your head

Imagine four boxes of books. Repartition is like emptying books onto a sorting table and rebuilding boxes according to a new rule or count. Coalesce is like combining neighboring boxes into fewer boxes without a full resort when Spark can do so. The final box count may match, but the work taken to get there can differ.

Words before we use them

repartition = reshuffle into a requested partitioning; coalesce = reduce partitions with limited movement where possible; narrow dependency = child partition depends on a small set of parent partitions; parallelism = tasks that can potentially execute concurrently; output-file count = often related to partition count at write time, not identical in every sink.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Partition-count changes in practical Spark work [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: DataFrame.repartition Hash repartitioning and partition count [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.repartition.html>

Official: DataFrame.coalesce Reduce partitions with a narrow dependency where possible [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.coalesce.html>

Follow with me once

Run the reasoning simulator --section partitions, then learner-PC 05_repartition_coalesce.py. The lab explicitly repartitions by region and then coalesces. Inspect both partition counts and formatted plans. Do not conclude that 4 or 2 is 'best'; the fixture is only 18 rows.

```
python practice/simulators/distributed_reasoning_simulator.py --section partitions
cd practice/spark && python 05_repartition_coalesce.py
```

What you should see

Both routes can report the requested final counts, but the repartition plan should make redistribution visible while coalesce reduction can use a narrower path. Exact plan nodes can evolve; explain the semantic distinction, not a brittle string match.

Why it worked

Parallel work is partition-driven, so too few partitions can underuse available cores and too many tiny partitions can create scheduling/file overhead. Repartition is justified when distribution/key layout needs to change; coalesce is useful when you mainly need fewer partitions after a shrinking operation.

Break it / common failure

repartition(1) before every write is a classic anti-pattern: it serializes output through one partition and can overload one task. The opposite mistake is forcing thousands of partitions because 'more parallelism is faster.' On an 18-row lab both choices can look harmless; production evidence is what matters.

Reason through a realistic incident

A daily output produces 8,000 tiny Parquet files. The team adds coalesce(1) and now one task runs for an hour. Diagnose both extremes. Estimate target file sizes/data volume, choose a reasonable output partition strategy, and validate total rows/keys after changing physical layout.

Deepen the engineering reasoning

Partitioning has logical and physical meanings. repartition('region') changes how keys are distributed for downstream work; filesystem partitioning such as partitionBy('order_date') creates directory layout at write time. They are related but not synonyms. Keep the distinction clear before M13 lakehouse design.

Try it yourself

For a 200 GB output on a cluster, explain what evidence you would collect before choosing a target partition/file count. Include total bytes, current partitions, task duration/skew and downstream read pattern. Do not give one magic number.

Fresh changed retry

After a strong filter leaves 2% of rows, the DataFrame still has hundreds of partitions. Compare coalesce versus repartition for the final write and state when the cheaper narrow reduction is attractive.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Change partitioning for a stated reason, inspect the plan/runtime effect, and revalidate rows/keys/totals. Physical tuning must not silently change business semantics.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-08

Aggregations and shuffle cost

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

groupBy/shuffle context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Aggregations look simple in SQL, but distributed grouping is often a network problem. Records with the same key must meet so Spark can form complete groups. That usually means shuffle/exchange. The right response is not to avoid all groupBy operations; it is to know when they occur, reduce unnecessary data before them and measure the cost.

Picture it in your head

Imagine invoices scattered across branches. To total by region, each branch can compute partial totals locally, but eventually all partials for East must meet, all North partials must meet, and so on. The shuffle is the redistribution step. Spark can combine locally where possible, yet the cross-partition key meeting still defines the expensive boundary.

Words before we use them

aggregation = summarize rows; partial aggregation = precombine within a partition before final combine; shuffle = network/disk redistribution across partitions; Exchange = plan marker for redistribution; shuffle partition = partition of post-shuffle output; spill = intermediate data written to disk when memory is insufficient.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)
groupBy/shuffle context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark SQL performance tuning Partition tuning and shuffle-aware optimization [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

Follow with me once

Run `distributed_reasoning_simulator.py --section shuffle` to prove the business totals first, then `learner-PC 06_aggregation_shuffle.py`. The source has 16 Completed rows totaling 25,230. Region totals must be East 5,950, North 6,380, South 6,000 and West 6,900. Inspect the Exchange and then reconcile the four results back to 25,230.

```
python practice/simulators/distributed_reasoning_simulator.py --section shuffle
```

```
cd practice/spark && python 06_aggregation_shuffle.py
```

What you should see

The grouped totals match the independent Python validator exactly. The Spark plan shows redistribution appropriate to a keyed aggregation. The four-row output is not evidence that only four records were processed.

Why it worked

Filtering to Completed before grouping reduces the population from 18 to 16. Projecting only needed columns can reduce data width. Those correctness-preserving reductions occur before an expensive boundary. But do not manually micro-optimize without measuring: Catalyst can already push many filters/projections.

Break it / common failure

Two dangerous shortcuts are increasing shuffle partitions blindly and collecting raw rows to aggregate in Python. The first may add overhead; the second defeats distributed execution and risks driver memory. Another error is changing the aggregation grain while 'optimizing', such as grouping by region+category when the requested output is region only.

Reason through a realistic incident

One grouped report suddenly produces 10x more shuffle bytes after a schema change. Check whether new wide columns are carried unnecessarily, whether the key cardinality changed, whether an upstream join multiplied rows, and whether a filter stopped pushing down. Do not tune partition counts before verifying the input population.

Deepen the engineering reasoning

Aggregation performance depends on data volume, key cardinality, distribution, row width and cluster resources. Explain plan tells you structure; SQL UI/runtime metrics tell you bytes/rows and task imbalance. Correctness controls remain simple: source population count, output grain, sum of groups versus independent overall total.

Try it yourself

Design an aggregation to compute Completed gross sales by category. State where a shuffle is likely, which columns are needed before it, expected two-row output and the independent 25,230 reconciliation.

Fresh changed retry

The business asks for top 100 customers by sales. Explain why `groupBy customer -> sum -> orderBy desc -> limit 100`` can involve more than one expensive distributed boundary and what you would inspect.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Do not fear shuffle; make it intentional. Reduce irrelevant data early, preserve the requested grain, inspect Exchange/runtime evidence and reconcile totals after every distributed aggregation.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-09

Joins and broadcast strategy

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Join strategy context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Joins are where Spark correctness and performance meet. Large-large joins often require repartitioning by join key. If one side is truly small enough, broadcasting it can send a copy to executors so the large side avoids a shuffle for that join. Spark can choose broadcast automatically from statistics, or you can use a hint deliberately; neither removes the need to validate keys and cardinality.

Picture it in your head

Picture a huge stack of orders at many worktables and a tiny customer lookup card. Shuffling both stacks by customer is expensive. If the lookup card is tiny, give a copy to every worktable and let each partition match locally. That is the broadcast idea. But broadcasting a 'lookup' that grew to gigabytes turns the shortcut into a memory problem.

Words before we use them

broadcast join = replicate small relation to executors; sort-merge join = distributed join strategy requiring compatible partitioning/sorting; join hint = request to optimizer; statistics = size/cardinality information used in planning; autoBroadcastJoinThreshold = size threshold for automatic broadcast eligibility; build side = relation used to build hash structure.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Join strategy context [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Spark SQL performance tuning Automatic broadcast, join hints and runtime strategy [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

Official: functions.broadcast Explicit broadcast hint API [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.functions.broadcast.html>

Follow with me once

Run `learner-PC 07_join_broadcast.py`. The 16-row Completed fact joins an eight-row unique customer dimension using `F.broadcast(dim)`. Inspect the formatted plan for broadcast-hash behavior and confirm the joined row count remains 16. Then read the current performance-tuning doc: Spark's default auto-broadcast threshold is 10 MB, but a hint or threshold is a planning tool, not proof the join is logically correct.

```
cd practice/spark
python 07_join_broadcast.py
```

What you should see

With a unique dimension, the left/fact grain remains one row per Completed order: 16 rows. The plan should show a broadcast join strategy in this tiny lab. Real automatic choices depend on statistics, size, AQE and configuration, so production strategy must be verified from the actual plan.

Why it worked

Broadcast can avoid shuffling the large relation because each executor has local access to the small lookup. The trade-off is replicated memory/network cost for the broadcast side. The engineering decision therefore depends on real size and reuse, not table naming conventions such as 'dimension'.

Break it / common failure

Never broadcast because a table is called `dim_customer`. Check size. Never use a broadcast hint to hide a many-to-many join. And do not assume the same join strategy is optimal forever: a table can grow, stats can become stale and AQE can adapt decisions at runtime.

Reason through a realistic incident

A lookup table was 2 MB last month and is now 800 MB after adding history columns. An old broadcast hint remains. Executors begin OOMing. Remove/reevaluate the hint, inspect size/statistics and consider a different join strategy or thinner projection. Then confirm row/key totals because performance repair must not alter semantics.

Deepen the engineering reasoning

Join planning has two independent questions: 'Should these rows match?' and 'How should Spark physically execute the match?' Solve logical keys/cardinality first. Then inspect size/statistics and plan. Mixing the questions leads to dangerous fixes, such as using `dropDuplicates()` to make a join fast when duplicate keys actually represent real history.

Try it yourself

Given 2 TB of fact data and a 5 MB static country-code lookup, explain why broadcast is a candidate. List three checks before hinting it and two runtime signs that the choice is unhealthy.

Fresh changed retry

The lookup becomes 60 MB compressed on disk but 300 MB in memory. Explain why file size alone is a poor basis for a broadcast decision and which Spark statistics/runtime evidence you

would inspect.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Validate join grain and key uniqueness before tuning. Treat broadcast as a strategy backed by size/plan evidence, not a rule attached to dimension tables.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-10

Join correctness, fan-out and reconciliation

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Join mechanics; book adds grain/fan-out reconciliation

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

A Spark join can run successfully, use an efficient broadcast, and still be financially wrong. Fan-out happens when a join key matches multiple rows on one or both sides. Distributed engines make wrong joins faster too. Therefore every join lesson must carry forward M03/M07 grain and reconciliation discipline.

Picture it in your head

Our Completed orders are one row per order. `customer_contacts_bad.csv` is one row per contact method, so C02 and C06 each appear twice. Joining orders to contacts on `customer_id` changes the grain to order-contact. The engine has done exactly what you asked; the duplicate money is a modeling error, not a Spark bug.

Words before we use them

fan-out = one source row becomes multiple joined rows; cardinality = relationship multiplicity; key uniqueness = one row per key where required; reconciliation = compare result counts/totals to independent controls; semi join = keep left rows with a match without adding right columns; anti join = keep left rows without a match.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Join mechanics; book adds grain/fan-out reconciliation [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: DataFrame API Join is an API operation; correctness still depends on keys/cardinality [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/dataframe.html>

Follow with me once

First run `distributed_reasoning_simulator.py --section join`. It computes duplicate contact keys and the row multiplication without Spark. Then learner-PC

`08_join_reconciliation.py` prints fact count, joined count, fact gross and joined gross. The correct Completed fact baseline is 16 rows / 25,230 gross. Any larger joined money proves repeated measures.

```
python practice/simulators/distributed_reasoning_simulator.py --section join
cd practice/spark && python 08_join_reconciliation.py
```

What you should see

The bad contact join has more than 16 rows because C02/C06 duplicate. Its gross-sales sum exceeds 25,230. That is expected failure evidence. The correct repair depends on the business requirement: aggregate contacts to customer grain, choose a valid current contact by explicit rule, or keep order-contact grain and never sum order-level measures as if still one row/order.

Why it worked

Spark's join operator obeys relational matching. If a left key has L matches and right key has R matches, that key can generate LxR joined combinations. Reconciliation catches the result even when every individual row looks plausible. This is why `count`, `countDistinct(order_id)` and `sum(gross_sales)` belong together after sensitive joins.

Break it / common failure

Do not use `dropDuplicates()` after the join as a universal fix: which contact row survives can be arbitrary and you may discard legitimate facts. Do not use `sum(distinct gross_sales)` either; two genuine orders can have the same amount. Repair at the grain/key rule, not at the appearance of duplicated values.

Reason through a realistic incident

A revenue dashboard doubles only for customers with multiple addresses. Spark jobs and plans look healthy. Trace dimension uniqueness by `customer_id`, calculate joined rows per order, and compare 25,230 source gross to post-join gross. The incident is a cardinality contract failure, not a shuffle tuning issue.

Deepen the engineering reasoning

Distributed joins add one more validation dimension: partition/skew performance. Keep it separate. First prove correctness at small scale with deterministic fixture; then test physical behavior at realistic size. A join can be logically correct but skewed, or fast but wrong. Your evidence should say which problem you are solving.

Try it yourself

Repair the contact join three ways on paper: (a) output one row/customer before joining, (b) preserve order-contact grain and remove order-gross aggregation, (c) use a semi join if only existence matters. State which business question each answers.

Fresh changed retry

A product dimension unexpectedly has two active rows for P09. Describe a pre-join uniqueness test and a post-join count/amount reconciliation that would stop the pipeline before serving wrong totals.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Carry relational discipline into Spark. Every important join should have declared left/right grain, expected cardinality, unmatched policy, key checks and at least one row/key/measure reconciliation.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-11

Data skew

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Uneven partitions and skew context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Data skew means work is not distributed evenly. A job can have hundreds of healthy tasks and one straggler holding most rows for a hot key. Adding executors or partitions does not automatically fix a key that inherently owns most of the data. Diagnose distribution first, then choose a remedy that preserves semantics.

Picture it in your head

Imagine 1,000 packages assigned by destination: 900 go to one warehouse named HOT while 100 are spread across ten others. Even with many trucks, the HOT warehouse's queue dominates. Our `skew_keys.csv` deliberately creates that 90% hot-key shape so you can see the problem before involving Spark.

Words before we use them

skew = uneven data/work distribution; hot key = key owning disproportionate rows; straggler = unusually slow task; salting = deliberately split a hot key into subkeys, then recombine correctly; skew join = join where key distribution creates uneven partitions; task duration distribution = runtime clue for imbalance.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Uneven partitions and skew context [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Spark SQL performance tuning Skew handling and AQE skew optimization [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

Follow with me once

Run the skew simulator, which proves 900/1000 rows use HOT. Then `learner-PC 09_skew_aqe.py` groups by key and explains the plan. In the Spark UI, compare task durations and input/shuffle sizes. The tiny lab may finish instantly; the concept is the distribution evidence, not elapsed time.

```
python practice/simulators/distributed_reasoning_simulator.py --section skew
```

```
cd practice/spark && python 09_skew_aqe.py
```

What you should see

The pure-Python control reports a 0.9 HOT share. The Spark output shows HOT=900. If task-level metrics are too small to look dramatic locally, record that honestly. You have proved the skew shape, not a production slowdown.

Why it worked

Hash partitioning sends equal keys together by design, which is exactly what grouping/join correctness needs. That means a dominant key can dominate one partition. Some remedies change execution strategy (AQE skew splitting), some change data shape (salting), and some change the business model (process giant tenants separately).

Break it / common failure

Random repartition without a key can redistribute rows for one stage but the next groupBy/join on HOT must bring the key back together. Salting without a second-stage recombination changes the result. And treating the longest task as skew without checking data distribution can confuse slow I/O or GC with hot keys.

Reason through a realistic incident

A nightly customer aggregation has median task time 20 seconds and one task at 18 minutes. The customer_id distribution shows one marketplace tenant owns 35% of rows. Explain why adding 10x partitions may still leave one huge key and propose evidence-led options: AQE skew handling, tenant

isolation, or two-stage salted aggregation where semantics permit.

Deepen the engineering reasoning

Skew is about distribution relative to work. A key with 1 million rows may be fine in a 1-billion-row job; a key with 100k could dominate a 120k-row job. Track proportions and bytes, not raw row count only. Check skew before and after joins because a fan-out can create artificial hot keys.

Try it yourself

Design a skew diagnostic checklist using key frequency, partition size, task duration and shuffle bytes. Then explain how you would verify a salting strategy still reconciles to the unsalted total.

Fresh changed retry

The HOT key is required to be processed as one logical group for a non-associative operation. Explain why naive salting may be invalid and what architecture alternatives you would consider.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Diagnose skew from distributions and runtime metrics. Choose a remedy that preserves the aggregation/join semantics, and prove both performance change and unchanged business totals.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-12

Adaptive Query Execution (AQE)

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Adaptive execution context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Adaptive Query Execution lets Spark revise parts of a SQL/DataFrame physical plan using runtime statistics. In Spark 4.2.0 AQE is enabled by default. It can coalesce small post-shuffle partitions, change supported join strategies and optimize some skewed shuffle partitions. AQE is powerful, but it is not an excuse to ignore data modeling or bad joins.

Picture it in your head

Picture a navigation app. The driver starts with a route based on estimates, then learns that one road is empty and another is jammed. It can reroute mid-journey. The destination and business request stay the same; the physical route changes based on observed conditions.

Words before we use them

AQE = adaptive query execution; runtime statistics = measurements from completed stages; post-shuffle partition coalescing = combine small shuffle partitions; adaptive join conversion = choose a different supported join strategy from runtime size; skew optimization = split supported oversized shuffle partitions; initial plan = pre-runtime physical choice.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Adaptive execution context [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Spark SQL performance tuning AQE, coalescing post-shuffle partitions and adaptive joins [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

Follow with me once

Run `distributed_reasoning_simulator.py --section aqe`, then inspect `spark.sql.adaptive.enabled` in `09_skew_aqe.py`. Read the 4.2.0 performance guide and note that AQE can coalesce post-shuffle partitions and adapt join choices. On the learner PC compare formatted plans and SQL UI details for a job with AQE on/off only as a controlled experiment; do not change five settings at once.

```
python practice/simulators/distributed_reasoning_simulator.py --section aqe
```

```
# learner PC: inspect spark.conf.get('spark.sql.adaptive.enabled')
```

What you should see

The course profile reports AQE true. You should be able to explain at least three adaptive capabilities and one limit: AQE reacts within supported plan structures; it does not repair wrong keys, missing filters, incorrect grain or every skew problem.

Why it worked

Estimates before execution are imperfect. Runtime map-output statistics reveal actual partition sizes and relation sizes, allowing Spark to make safer choices. This reduces the need to guess a perfect static shuffle-partition count for every stage, while still requiring a sensible initial plan and resource envelope.

Break it / common failure

Do not say AQE 'automatically makes Spark fast.' It can add planning benefits only where supported and may leave root causes untouched. Do not compare elapsed time from one tiny local run as proof. Warm caches, JVM startup and background load can dominate classroom timings.

Reason through a realistic incident

A sort-merge join plan begins with uncertain size estimates; at runtime one side is much smaller than expected and AQE converts the join. Explain why this can help, but still check joined row count and money total. Physical adaptation must not change semantic validation.

Deepen the engineering reasoning

AQE shifts some tuning from fixed compile-time guesses to runtime decisions. That changes how you read plans: distinguish initial/optimized plan concepts from final adaptive plan/runtime statistics. In interviews, explain the capability and evidence; do not promise a specific conversion without seeing the actual query/runtime.

Try it yourself

Write a controlled AQE experiment plan: same input and query, AQE true versus false, capture plan, task/partition metrics and exact business output. List confounders you would hold constant.

Fresh changed retry

Suppose AQE coalesces many tiny post-shuffle partitions but one hot key still dominates a task. Explain why both facts can be true and what next evidence you need.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Keep AQE enabled unless you have evidence-based reasons otherwise, but treat it as one layer of execution optimization. Validate data distribution, join correctness and output invariants independently.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-13

Caching and persistence

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Repeated-action/caching context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Caching is useful when the same expensive DataFrame is reused across multiple actions, but it consumes executor memory and can make a job worse if the data is too large, used once, or never unpersisted. The right question is not 'is cache fast?' but 'will avoiding repeated recomputation repay materialization/storage cost under this workload?'

Picture it in your head

Imagine photocopying a long reference packet. If ten people will consult it repeatedly, one shared copy can save repeated trips to the archive. If you need one page once, making and storing the full copy wastes time and shelf space. Spark cache is that trade-off for computed partitions.

Words before we use them

cache = persist DataFrame with default storage level; persist = choose storage level; materialize = execute partitions so cached blocks exist; unpersist = release cached blocks; lineage recomputation = rebuild data from dependencies; storage tab = Spark UI view of persisted datasets.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Repeated-action/caching context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: DataFrame.cache Persist a DataFrame for reuse [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.cache.html>

Official: Spark SQL performance tuning Caching behavior and memory trade-offs [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

Follow with me once

Run the pure simulator cache section, then learner-PC 10_cache_persistence.py. The script caches the Completed+gross DataFrame because it is reused for region and category summaries, triggers count to materialize it, runs both downstream actions, then calls unpersist. Inspect the Storage tab if available.

```
python practice/simulators/distributed_reasoning_simulator.py --section cache
cd practice/spark && python 10_cache_persistence.py
```

What you should see

The business results still reconcile to 25,230. Cache should appear only after an action materializes it. After unpersist, the explicit cache is released. On this tiny dataset elapsed time is not meaningful evidence; the lab demonstrates lifecycle and reuse reasoning.

Why it worked

Without persistence, separate actions may replay the upstream lineage. Caching stores computed partitions so reuse can avoid some recomputation. But blocks occupy memory/disk according to storage level and may be evicted. The benefit depends on cost to recompute, reuse count, size and available memory.

Break it / common failure

Caching every intermediate DataFrame is an anti-pattern. It can crowd useful data, increase GC and make lineage harder to reason about. Another mistake is calling cache without any action and claiming data is already stored. Cache is lazy too.

Reason through a realistic incident

An analyst caches five large DataFrames in a notebook and never unpersists them. Executors spend more time in GC and spill. Remove caches one by one, identify actual reuse, inspect Storage/runtime metrics and preserve only evidence-backed persistence decisions.

Deepen the engineering reasoning

Persistence is a performance contract, not a correctness contract. Your output should not change because of cache. This makes A/B validation straightforward: same row/key/totals with cache off/on. If results change, investigate nondeterministic input/UDF/side effects rather than celebrating a faster run.

Try it yourself

For three cases - single final write, DataFrame reused by six aggregates, and tiny lookup broadcast repeatedly - decide cache/no-cache and state what runtime evidence would support the choice.

Fresh changed retry

A cached DataFrame is 60 GB but executors have limited memory. Explain how spill/eviction can reduce benefit and why MEMORY_ONLY thinking from old tutorials may not fit the default DataFrame cache behavior.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Cache intentionally, materialize knowingly, inspect actual reuse/storage, and unpersist when the reuse window ends. Never use cache as a reflexive line in every pipeline.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-14

Parquet I/O and partition pruning

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Parquet read/write and partitioned data context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Spark and Parquet complement each other: Parquet supplies typed columnar storage and metadata, while Spark can request only required columns, apply filters and exploit partition directory layout. But 'partition pruning' is not magic. It depends on the written layout and query predicates, and you should inspect plans/files rather than claiming every filter skips data.

Picture it in your head

Picture a warehouse whose aisles are labelled by order_date and whose boxes are columnar. If a query asks only Sept 3 and only order_id/region/gross_sales, Spark can avoid unrelated date directories and unnecessary columns when the data source supports it. If files are badly partitioned or the predicate is wrapped in an opaque UDF, that advantage can disappear.

Words before we use them

Parquet = columnar file format; column pruning/projection = read needed columns; predicate pushdown = push filter toward data source when supported; partition pruning = skip filesystem/table partitions based on partition filters; partition discovery = infer partition columns from directory paths; file scan = physical plan operator reading files.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Parquet read/write and partitioned data context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark Parquet data source Parquet read/write/schema/partition discovery behavior [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-data-sources-parquet.html>

Follow with me once

On the learner PC run 11_parquet_pruning.py. It writes the 18-row fixture partitioned by order_date, then reads only four columns for 2026-09-03 and calls explain. Inspect the output directory and plan. This tiny write intentionally produces educational files, not an optimized production layout.

```
cd practice/spark
```

```
python 11_parquet_pruning.py
```

What you should see

The read returns only rows for 2026-09-03 and the requested columns. The plan should show Parquet/file-scan information including partition filters when applicable. Real Spark/Parquet runtime evidence must be collected on the learner PC; static checks alone do not prove scan behavior.

Why it worked

Physical layout gives the engine opportunities to avoid work, but Spark still decides a plan. Projection reduces column I/O; partition filters can avoid whole directory partitions; Parquet statistics can help skip lower-level chunks. These are distinct mechanisms and should not be collapsed into one slogan.

Break it / common failure

Partitioning by high-cardinality `order_id` would create tiny directories/files and metadata overhead. Another error is thinking where `(year(order_date)==2026)` always prunes a directory partition exactly like a direct partition-column predicate; verify the plan. Do not infer pruning from fast execution on 18 rows.

Reason through a realistic incident

A query filtering one day scans a month of data. Check the physical directory/table partitioning, predicate form, partition-column type, plan's PartitionFilters and file counts. The problem may be layout, expression shape or simply that the table was never partitioned by that field.

Deepen the engineering reasoning

M11 distinguished row-group skipping from filesystem partitioning. M12 adds distributed read planning. Keep layers separate: directory/table partition pruning, Parquet column projection, Parquet predicate/statistics behavior and Spark task/file scheduling can all reduce different work. Evidence should identify which layer actually applied.

Try it yourself

Design a Parquet layout for two years of daily orders. Compare partitioning by date, region, `customer_id` and `order_id`. State the likely query pattern and small-file risk behind your choice.

Fresh changed retry

The dominant query shifts from daily date slices to customer-history lookups. Explain why the old physical layout may no longer be ideal and why you should measure before repartitioning historical data.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Choose partition layout from stable access patterns and volume, keep file sizes reasonable, and verify pruning/projection in the actual plan. Logical output checks stay identical regardless of physical layout.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-15

Built-in functions vs Python UDFs

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Built-in expression style and transformation examples

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Spark's built-in SQL/DataFrame functions are generally preferable to ordinary Python UDFs because Spark can understand built-in expressions inside its optimized plan. A Python UDF introduces a Python execution boundary and hides the function's internals from Catalyst. UDFs remain valid when the transformation truly cannot be expressed with appropriate built-ins, but they should be a deliberate exception.

Picture it in your head

Imagine giving the planner a standard instruction card versus a sealed envelope. upper, when, arithmetic and date functions are standard cards Spark recognizes and can reason about. A Python UDF is a sealed envelope: Spark knows input/output types but not the internal Python logic well enough to optimize through it.

Words before we use them

built-in function = Spark SQL expression implemented/understood by engine; Python UDF = user-defined Python function applied through Spark UDF boundary; Catalyst = optimizer; serialization = convert data between runtime representations; pandas UDF = vectorized UDF route with different execution characteristics; expression tree = structured representation of built-ins.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Built-in expression style and transformation examples [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: PySpark SQL functions Optimizer-visible built-in functions catalogue [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/functions.html>

Follow with me once

Run learner-PC 12_builtin_vs_udf.py. It creates the same uppercase region result once with F.upper and once with a Python UDF, then prints both plans. Focus on the presence of a Python-evaluation node in the UDF plan. Do not use this tiny job's elapsed time as a universal benchmark.

```
cd practice/spark
python 12_builtin_vs_udf.py
```

What you should see

Both versions can produce the same uppercase values. The built-in plan stays within Spark SQL expression machinery; the Python UDF plan exposes a Python UDF evaluation boundary. This is a plan-level teaching observation, not a promise of a fixed speed ratio.

Why it worked

Built-ins give Spark richer semantics for optimization, code generation and execution. Python UDFs require data to cross into Python workers and reduce what Catalyst can see. That overhead matters especially at scale, but correctness still comes first: use the expression that accurately represents the rule.

Break it / common failure

Do not contort a clear transformation into unreadable built-ins merely to avoid every UDF. Also do not define a UDF for simple tasks Spark already supports, such as uppercasing, date extraction or conditional columns. And never forget a return type/NULL behavior contract.

Reason through a realistic incident

A pipeline spends much of its time in PythonUDF evaluation for a function that only trims and uppercases text. Replace it with trim/upper, compare plans and outputs, then measure at realistic scale. Save an equality check so the optimization does not change cleaning semantics.

Deepen the engineering reasoning

The choice is part of portability and observability too. Built-in SQL expressions are easier for other Spark languages/tools to inspect. If you need a UDF, isolate it, document deterministic/null/error behavior, test it on edge cases and consider whether a vectorized alternative fits.

Try it yourself

Review five transformations - uppercase, regex replace, date difference, custom cryptographic library call, complex proprietary scoring function. Mark built-in first versus UDF candidate and explain why.

Fresh changed retry

A Python UDF sometimes throws on null input. Design a test and repair contract without silently replacing null with a business value that was never specified.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Prefer optimizer-visible built-ins when they express the rule clearly. Use UDFs for genuine gaps, with explicit types, null/error behavior, tests and plan/runtime evidence.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-16

Driver safety: collect, toPandas and bounded inspection

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Driver vs distributed work context

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

The Spark driver is not a free dumping ground. `collect()` returns all rows to the driver process; `toPandas()` similarly materializes the `DataFrame` as a pandas object on the driver. Both can be appropriate for deliberately small bounded results, but calling them on unknown-large data can crash the application even when executors have plenty of memory.

Picture it in your head

Picture a distributed warehouse with many shelves but one office desk. Spark can store/process boxes across shelves, yet `collect()` tells every shelf to send its selected contents to that one desk. If you know the result is five rows, fine. If you request a billion rows, the desk is the bottleneck.

Words before we use them

driver OOM = driver runs out of memory; `collect` = return all `DataFrame` rows as Python list; `toPandas` = convert all data to pandas `DataFrame` in driver memory; `first/take/head` = bounded row retrieval APIs; `limit` = logical bound before retrieval; distributed write = keep output distributed instead of returning to driver.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Driver vs distributed work context [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: `DataFrame.collect` Returns all rows to the driver; use deliberately [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.collect.html>

Official: `DataFrame.toPandas` Loads data into driver memory; intended for small results [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.DataFrame.toPandas.html>

Follow with me once

Run the driver simulator, then learner-PC `13_driver_safety.py`. It performs the gross-sales aggregate in Spark and retrieves one scalar with `first()`. For row inspection it uses `orderBy(...).limit(5).collect()`. Compare this deliberate bound with the commented warning against `df.collect()` on unknown-large data.

```
python practice/simulators/distributed_reasoning_simulator.py --section driver
cd practice/spark && python 13_driver_safety.py
```

What you should see

The aggregate returns 25,230 and the bounded inspection returns at most five Row objects. No full 18-row collect is necessary even in the toy lab because we are building the habit that scales. A small result after a distributed aggregation is different from dragging raw data to Python for computation.

Why it worked

Driver memory is separate from total cluster capacity. Distributed operations keep partitions on executors; collection centralizes them. The safest pattern is reduce/filter/aggregate/write in Spark, then bring back only a known-small diagnostic or scalar result.

Break it / common failure

`df.limit(5).toPandas()` is bounded; `df.toPandas().head()` is not, because the full DataFrame reaches the driver before pandas takes the head. Likewise, writing `len(df.collect())` to count rows is far worse than `df.count()`. API order matters.

Reason through a realistic incident

A notebook works on sample data but crashes in production at `toPandas()` after source volume grows 100x. Move aggregations to Spark, keep only a bounded summary for pandas/plotting, and put an explicit size/row contract before any driver conversion.

Deepen the engineering reasoning

Driver safety also covers Python-side loops, huge `collect_set` results, oversized broadcast variables and logging giant records. Ask where each result lives. If the answer is 'one process', define a bound. This reasoning is more durable than memorizing a blacklist of functions.

Try it yourself

Rewrite three unsafe patterns: `len(df.collect())`, `df.toPandas().groupby(...)`, and looping over all collected rows to write files. Use distributed Spark equivalents and state any tiny bounded result you still retrieve.

Fresh changed retry

You need to email a 50-row quality report. Explain a safe Spark path that proves the result is ≤ 50 rows before converting to pandas, and what you would do if the bound fails.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Keep large data distributed. Materialize on the driver only after a proven bound or when the API inherently returns a small scalar. Treat driver-memory assumptions as explicit contracts.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-17

Testing and debugging Spark transformations

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Transformation refresher; testing task remains independent

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

Spark transformations should be tested like other engineering logic, but distributed unordered results create special traps. Tests need explicit schemas, small deterministic fixtures and invariant-oriented assertions. PySpark 4.2.0 provides `assertDataFrameEqual` and `assertSchemaEqual`, which can integrate with `unittest`/`pytest`; ordering must be deliberate when row order matters.

Picture it in your head

Think of a wind tunnel for a pipeline: tiny controlled data exposes the same transformation rules without production scale. You do not test Spark by staring at `show()` output. You feed known rows, run one transformation and compare schema/rows/invariants to an expected result.

Words before we use them

unit test = small isolated behavior test; fixture = controlled input; `assertDataFrameEqual` = compare DataFrame content; `assertSchemaEqual` = compare schema; deterministic ordering = explicit sort when test requires order; invariant = property that must remain true, such as unique `order_id` and conserved total.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range)

Transformation refresher; testing task remains independent [OPEN SOURCE - CLICK HERE](#)

<https://www.youtube.com/watch?v=ti3aC1m3rE8>

Official: Testing PySpark `assertDataFrameEqual`/`assertSchemaEqual` and test-framework patterns [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/testing_pyspark.html

Follow with me once

On learner PC run `14_testing_debugging.py`. It computes region totals from the 18-row fixture and compares them to an expected four-row DataFrame. The test sorts both results because DataFrame row order is otherwise not guaranteed. Then deliberately change one expected amount and read the assertion diff before restoring it.

```
cd practice/spark
python 14_testing_debugging.py
```

What you should see

The correct test prints PASS. The schema and four rows match. When you inject an incorrect expected value, the testing utility identifies mismatching rows rather than relying on visual inspection. This is stronger evidence than a screenshot of `show()`.

Why it worked

Tiny tests catch transformation logic before cluster cost and data volume hide the problem. They also make refactoring safer: a change to join/partition strategy should preserve logical invariants. Runtime/performance tests are separate; a unit test cannot prove production scaling.

Break it / common failure

Do not assert incidental row order unless the transformation contract includes ordering. Do not use the same buggy production function to generate 'expected' data. And do not replace tests with one final row count: a fan-out can be followed by a filter that returns the expected count but wrong identities/totals.

Reason through a realistic incident

A refactor from SQL to DataFrame API passes row count but changes North total. A four-row expected DataFrame catches the difference immediately. Use a minimal failing fixture to isolate whether filter,

calculation, join or null handling changed.

Deepen the engineering reasoning

Layer tests: schema contract, row/key invariants, measure reconciliation and selected expected rows. For complex pipelines, keep a pure-Python or SQL independent control when practical so Spark implementation and expected result are not generated by the same logic. This module's `validate_expected.py` is that independent baseline.

Try it yourself

Write a test plan for the bad contact join: one assertion should intentionally prove the naive join fails cardinality, and a repaired join should pass unique order count and 25,230 gross reconciliation.

Fresh changed retry

A DataFrame contains floating calculations. Explain when approximate tolerance is appropriate and why loosening tolerance cannot be used to hide a true business discrepancy.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

Test transformations on tiny deterministic data, separate logical tests from performance tests, and save failed assertion evidence before repair. A plan refactor should not silently alter business invariants.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.

DE12-18

Integrated PySpark pipeline and distributed-processing checkpoint

Watch first

[1. Optional visual: DataTalksClub - Apache Spark / PySpark \(no verified bounded range\)](#)

Review the flow, then close video and build independently

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Why this matters

The final M12 checkpoint is not 'write some PySpark.' You must build and defend a repeatable pipeline that preserves the 18-row source contract, filters to 16 Completed orders, derives gross sales, reconciles 25,230, explains shuffle/join/partition choices, avoids unsafe driver collection, writes Parquet and has deterministic tests. Distributed engineering is correctness plus execution reasoning.

Picture it in your head

Picture the whole evidence chain: typed source -> filtered/derived DataFrame -> plan inspection -> grouped summaries -> safe reuse -> bounded inspection -> Parquet write -> independent controls/tests. Every arrow has a reason and a check. If you cannot explain which data moves across the network or which result lives on the driver, the pipeline is not yet defensible.

Words before we use them

distributed checkpoint = integrated proof of Spark semantics and business correctness; invariant matrix = list of counts/keys/totals that must hold; execution evidence = plan/UI/task/partition proof; logical evidence = rows/keys/totals/schema; reproducible job = script with pinned environment and deterministic inputs/outputs; handoff = enough instructions/evidence for another engineer to rerun safely.

Useful visual / current reference

Optional visual: DataTalksClub - Apache Spark / PySpark (no verified bounded range) Review the flow, then close video and build independently [OPEN SOURCE - CLICK HERE](#)

https://www.youtube.com/watch?v=r_Sf6fCB40c

Official: Spark SQL performance tuning Plan, partition, join, cache and AQE controls used in final checkpoint [OPEN SOURCE - CLICK HERE](#)

<https://spark.apache.org/docs/4.2.0/sql-performance-tuning.html>

Official: Testing PySpark Final pipeline tests and invariant checks [OPEN SOURCE - CLICK HERE](#)

https://spark.apache.org/docs/4.2.0/api/python/getting_started/testing_pyspark.html

Follow with me once

Read `15_integrated_pipeline.py` but do not copy it into your independent attempt. The reference filters Completed rows, derives gross_sales, groups by region/category, explains a grouped plan, validates region totals, demonstrates deliberate cache lifecycle, uses only bounded collect and writes partitioned Parquet. First run the pure-Python validator and integrated simulator so you know the independent source controls before Spark executes.

```
python practice/reference_validation/validate_expected.py
python practice/simulators/distributed_reasoning_simulator.py --section integrated
# learner PC reference only after your own attempt:
cd practice/spark && python 15_integrated_pipeline.py
```

What you should see

Source=18; Completed=16; Completed gross=25,230. Region East=5,950, North=6,380, South=6,000, West=6,900. Category Accessories=16,830, Office=8,400. Unique Completed order IDs=16. Your Spark plan/runtime evidence must be added on the learner PC; the build package supplies only static/simulator/reference validation.

Why it worked

The checkpoint combines all four M12 layers: business semantics, distributed API correctness, physical-plan reasoning and operational safety. None substitutes for the others. A perfect explain plan with wrong totals fails; correct totals produced by collecting everything to pandas also fail the distributed-engineering objective.

Break it / common failure

Common failure modes are implicit schema, accidental Pending/Cancelled inclusion, join fan-out, blind repartition(1), unbounded collect/toPandas, caching one-use data, claiming AQE/skew improvements without runtime evidence, and reporting the 25,230 control from memory instead of recomputing it from the supplied source.

Reason through a realistic incident

A code review asks: 'Why Spark for this tiny dataset?' The correct answer is that the dataset is intentionally tiny so you can inspect every business control while exercising real Spark APIs locally. You are learning transferable execution semantics; you are not claiming Spark is efficient for an 18-row production workload.

Deepen the engineering reasoning

Your handoff should separate static/package evidence (Python syntax, source controls and declared plan intent) from runtime evidence that only a real Spark session can prove: SparkSession startup, physical plans, UI jobs/stages/tasks, Parquet output, cache behavior, broadcast decisions and AQE behavior. This honesty boundary is itself production engineering discipline.

Try it yourself

Build `m12_independent_pipeline.py` from a blank file using the supplied CSVs and common session helper. Required evidence: explicit schema, local-safe session, Completed filter, gross calculation, region+category reconciliation, one explain output, one deliberate partition decision, bounded driver inspection, Parquet write, at least two DataFrame assertions, and a short note explaining any shuffle/broadcast/cache choice you made or deliberately did not make.

Fresh changed retry

Use `practice/assessment/practical/parcel_orders.csv` as a changed domain. Build a Spark summary by region for Completed parcels, prove total gross=6000 and completed rows=7, then perform a fresh retest on `clinic_jobs.csv` where Completed amount=3500 across 6 jobs. Do not reuse hard-coded region names or amounts from the order fixture.

Validation and recovery ladder

- State input grain, output grain and the exact business invariant before tuning execution.
- Inspect the smallest independent control first: source row/key totals, pure-Python validator or expected fixture result.
- Then inspect Spark structure: schema, partitions, explain plan, and on the learner PC the UI/runtime metrics.
- Repair one cause at a time. Do not hide a correctness failure with repartition, cache, broadcast hints or dropDuplicates.
- Open the Explained Review only after saving the genuine attempt; close it and solve the changed retry.

Ready-made revision

A Spark pipeline is mastered when you can rebuild it on changed data, prove output with independent controls, interpret the physical plan/runtime evidence, and explain resource/driver/partition safety. Save the full evidence packet; M13 then adds real Delta/lakehouse transaction semantics.

Evidence / mastery

Save code, actual result/error, one independent row/key/amount control, relevant plan/runtime evidence, a 60-90 second explain-back and the changed retry. If Spark itself was not executed, label the runtime part PENDING rather than converting static/simulator evidence into a fake runtime PASS.